



PROJECT AND TEAM INFORMATION

Project Title

Smart Driver Drowsiness & Safety System

<i>Team Name: Smart Safety Team</i> <i>Team #: DSCPP-III-2026-T234</i>	<i>PHOTO</i>
Team Member 1 (Team Lead) <i>Name: RIDVIK GARG</i> <i>University Roll No.: 2028039</i> <i>Section: H</i> <i>Email: ridvikridvik74@gmail.com</i>	
Team Member 2 <i>Name: Prabhav Gulyani</i> <i>University Roll No.: 2027975</i> <i>Section: E</i> <i>Email: Prabhav7424@gmail.com</i>	
Team Member 3 <i>Name: Nandini Gupta</i> <i>University Roll No.: 2028777</i> <i>Section: ML4</i> <i>Email: nandinigupta787@gmail.com</i>	
Team Member 4 <i>Name: Antriksh Gehlot</i> <i>University Roll No.: 2028648</i> <i>Section: ML3</i> <i>Email: antrikshgehlot9290@gmail.com</i>	

Mentor: Mr. Kamal Kumar Gola

PHASE-II PROJECT PROGRESS DESCRIPTION (35 pts)

Project Abstract

The main idea behind our Smart Driver Drowsiness & Safety System is to prevent accidents caused by sleepy or distracted drivers. We are capturing a live video feed from a webcam to track the driver's face. The system specifically looks at eye closures and head positions. It won't beep for every small movement. Instead, the code verifies if the eyes remain closed for a longer duration to confirm an actual threat.

We have designed a step-by-step alert mechanism. First, a normal beep plays. If the driver ignores it, a loud alarm triggers. We also planned a feature to reduce the car's music volume so the driver can hear the warning clearly. In extreme cases, an emergency SMS goes out.

After the Phase-I evaluation, we updated our code to properly implement Data Structures. The project now uses a Queue for incoming safety events, a Stack for recent alerts, and a Linked List to maintain the history. The entire logic is written in C++, with OpenCV handling camera inputs and SQLite managing the database.

Updated Project Approach and Architecture

We divided the project into separate modules. This makes it easier to test each part before we combine everything. The process starts with the camera capturing the driver's face. When our OpenCV logic spots prolonged eye closure, it generates a safety event and forwards it to the main application.

Based on our mentor's feedback in Phase-I, we changed how we handle data. We now apply DSA concepts directly. Incoming events are pushed into a Queue so they process in the right order. We manage the UI warnings using a Stack, which ensures the latest alerts show up on top. A Linked List is used to keep a record of all past events.

Once an event is handled, the backend decides the appropriate action. This could be triggering an alarm, pausing the music, or sending a text message. C++ runs the core operations, OpenCV takes care of video frames, and SQLite stores our data.

Tasks Completed

Task Completed	Team Member
Finalized the main requirements and split the project into smaller modules.	All Team Members
Updated the architecture to use Queues, Stacks, and Linked Lists.	Ridvik / Prabhav
Set up the C++ repository and got the development environment ready.	All Team Members
Wrote the initial code for handling safety events and maintaining logs.	Ridvik / Prabhav
Tested the webcam input and verified basic face tracking with OpenCV.	Prabhav / Nandini
Created the SQLite database and tested simple insert/read queries.	Nandini / Antriksh

Challenges/Roadblocks

- 1. Lighting conditions:** OpenCV struggles to track eyes if the room is dark or if the driver wears spectacles. We are experimenting with different Eye Aspect Ratio (EAR) values to fix this.
- 2. DSA implementation:** Replacing standard vectors with our own custom Queue and Stack classes took a lot of extra time and caused some bugs initially.
- 3. Module integration:** Linking the C++ backend with the OpenCV feed and database often leads to crashes. We are doing module-by-module testing to catch memory leaks early.
- 4. System lag:** Processing live video is heavy on the CPU. Dropping frames delays the alerts, so we have to optimize our code for better speed.
- 5. Realistic testing:** Faking drowsiness by blinking slowly isn't very accurate. We need to collect diverse test videos to see if the detection actually works in real life.

Tasks Pending

Task Pending	Team Member (to complete the task)
Adjust the eye-tracking logic and link it with the main event Queue.	Prabhav / Nandini
Finish debugging the custom Stack and Linked List classes.	Ridvik / Prabhav
Write the code to trigger different sound alerts based on severity.	Prabhav / Antriksh
Create a script to lower or pause background music during warnings.	Nandini / Antriksh
Set up the SMS API to send texts to emergency contacts.	Antriksh
Merge all the modules, find bugs, and conduct full testing.	All Team Members

Project Outcome/Deliverables (2 pts)

By the end of this semester, we plan to deliver a fully functional prototype of our smart safety system. Through webcam monitoring, the software will detect driver fatigue and respond with different actions like sounding an alarm, muting background music, or sending an SMS.

Apart from the actual software, this project is helping us understand how to apply standard Data Structures (like Stacks, Queues, and Linked Lists) inside a real-time application.

For our final college submission, we will provide the C++ source code, the OpenCV detection scripts, the SQLite database, our testing logs, the project report, and a public GitHub link containing all our files.

Progress Overview

We have completed the planning phase and revised our system architecture based on the Phase-I review. The core setup is done. The C++ environment compiles successfully, the camera feed works, and the database tables are ready. Recently, we focused on rewriting the event handler using Stacks, Queues, and Linked Lists.

At present, we are spending our time coding and fixing bugs. The eye tracking works fine under good lighting, but we are still adjusting it for low-light conditions. We also started working on the audio alerts and SMS integration.

Since we are developing the modules separately, a fully combined prototype is not ready yet. In the upcoming weeks, we will link the OpenCV triggers with our C++ backend to ensure the alerts work without delay.

Codebase Information

GitHub Repository:

<https://github.com/ridvik74/Smart-Driver-Drowsiness-Safety-System>

Repository Status: Public

Main Branch: main

To manage the code better, we split the tasks among the team. Some of us are working on the C++ logic, while others handle the database and camera scripts. Every member pushes their updates from their own GitHub account to keep a clear record of individual work.

Important development areas: Right now, the major development areas in our repository are the custom DSA headers (Queue, Stack, Linked List), the camera processing script, and the alert generation logic.

Testing and Validation Status

Test Type	Status	Notes
C++ Environment	Pass	Checked the build files; they compile fine on our laptops.
Webcam Capture	Pass	Camera turns on and OpenCV captures video without crashing.
DB Initialization	Pass	Database is ready. We can insert and fetch records.
DSA Integration	In Progress	Custom classes are done, but we are fixing some memory leaks.
EAR/Vision Logic	In Progress	Works in good light, still adjusting the code for darker rooms.
Integration Testing	Pending	Waiting for all modules to finish before final merging.

Deliverables Progress

Requirements & Planning: Done

Phase-I Architecture Redesign: Done

C++ Backend Setup: Working on it

DSA (Queue, Stack, Linked List) Logic: Working on it

OpenCV Vision Module: Working on it

SQLite Database: Working on it

Audio Alerts & Music Mute: Working on it

SMS Emergency Feature: Not started yet

Final System Integration: Not started yet

Live Environment Testing: Not started yet

Final Report & Documentation: Not started yet

GitHub Repo Maintenance: Ongoing (Public)